

26. Payment Gateway (Card Processing Core)

Interviewer: Design a **payment gateway** — the system a merchant integrates with to charge a customer's card. Money moves from the cardholder, through the card networks and banks, to the merchant. It has to be correct to the cent, never double-charge, survive failures mid-transaction, and satisfy compliance. Build the core.

Candidate: This is the problem where **"eventually consistent" is not good enough for the money, but the outside world (banks, card networks) is slow, flaky, and asynchronous** — so the whole design is about reconciling strong internal correctness with an unreliable external world. The words that will dominate: **idempotency, the double-entry ledger, state machines, and reconciliation**. Let me clarify.

1. Clarifying the requirements

Candidate: 1. Are we the **gateway/processor** talking directly to card networks (Visa/Mastercard) and acquiring banks, or a layer in front of a processor like Stripe? I'll assume we're the gateway orchestrating auth/capture against acquirers. 2. Scope: **authorization, capture, refund, void** — the full card lifecycle? 3. Do we **store card data** (PCI-DSS scope) or tokenize via a vault? Huge architectural implication. 4. Volume — transactions/sec at peak? 5. Multi-currency / multi-region?

Interviewer: You're the gateway, orchestrating against multiple acquiring banks. Support auth, capture, refund, void. **Tokenize cards** — you do not want raw PANs everywhere. Assume ~10,000 transactions/sec at peak, global, multi-currency.

Candidate: Good — tokenization keeps most of the system **out of PCI scope**, which I'll lean on. Requirements:

Functional - `authorize` (hold funds), `capture` (move held funds), `refund`, `void`. - Tokenize card → store token in a PCI vault; the rest of the system sees only tokens. - Route each transaction to the right acquiring bank/processor. - Idempotent operations (retries must never double-charge). - Webhooks/notifications to merchants on state changes. - An auditable ledger of every money movement.

Non-functional - **Correctness is absolute** — never double-charge, never lose a transaction, books must always balance. This is stronger than any other system in this playbook. - **Durability** — a confirmed payment must survive any crash. - **Availability** — merchants can't checkout if we're down; but we favor **consistency over availability** when they conflict (better to reject than to double-charge). - **Latency** — auth should feel synchronous to the buyer (~1–3s), bounded by the bank. - **Compliance** — PCI-DSS; audit trail; idempotency; strong isolation of card data.

2. Estimation

10,000 TPS peak. Each payment touches several state transitions + external calls.
auth call to bank: 500ms–3s (SLOW, external, unreliable) → must handle async/timeouts

Storage: say 50M transactions/day * ~5 KB (txn + ledger entries + audit) = ~250 GB/day.
Financial data is kept for YEARS (regulatory) → partition + archive, but always durable.

The scale challenge is modest (10k TPS is not huge); the CORRECTNESS + FAILURE-HANDLING challenge is the whole game. A bank call that times out is the norm, not the exception – did the charge go through or not? The design must answer that deterministically.

3. Idempotency – the absolute centerpiece

Interviewer: A merchant sends a charge, the network times out, they retry. How do you not charge twice?

Candidate: Every mutating request carries a client-supplied **Idempotency-Key**. This is non-negotiable in payments.

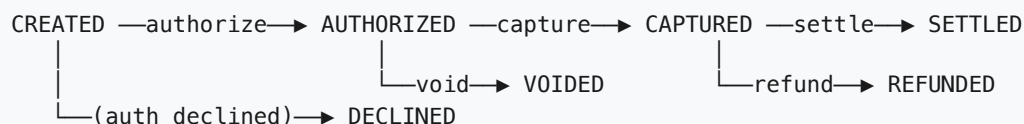
```
POST /payments  Idempotency-Key: <uuid from merchant>
├─ look up (merchant_id, idempotency_key) in the idempotency store
│   ├── FOUND + completed → return the STORED original response (no re-charge)
│   ├── FOUND + in-progress → return "still processing" / 409 (don't start a second)
│   └── NOT FOUND ↓
├─ atomically INSERT (merchant_id, idempotency_key, status=IN_PROGRESS)  -- unique key
│   (if the insert loses a race, treat as FOUND above)
├─ execute the payment (state machine + bank call)
└─ store the final response against the key; return it
```

- The (merchant_id, idempotency_key) pair has a **unique constraint** in a strongly consistent store — two concurrent retries can't both start; one wins the insert, the other reads the in-progress/complete record.
- Idempotency records are retained long enough to cover all realistic retries (e.g. 24h+).
- **This alone prevents the most catastrophic bug in payments** — the double charge.

4. The transaction state machine + double-entry ledger

Candidate: A payment is not a row you flip — it's a **state machine**, and every state change writes **immutable ledger entries**. Never UPDATE balance = balance - x; instead **append** entries and derive balances. That's how real financial systems stay auditable and correct.

Payment state machine:



Every arrow = an atomic transition + append-only ledger entries.

Double-entry ledger (the source of truth for money):

Every movement writes balanced entries (debits == credits). Example capture of \$100:

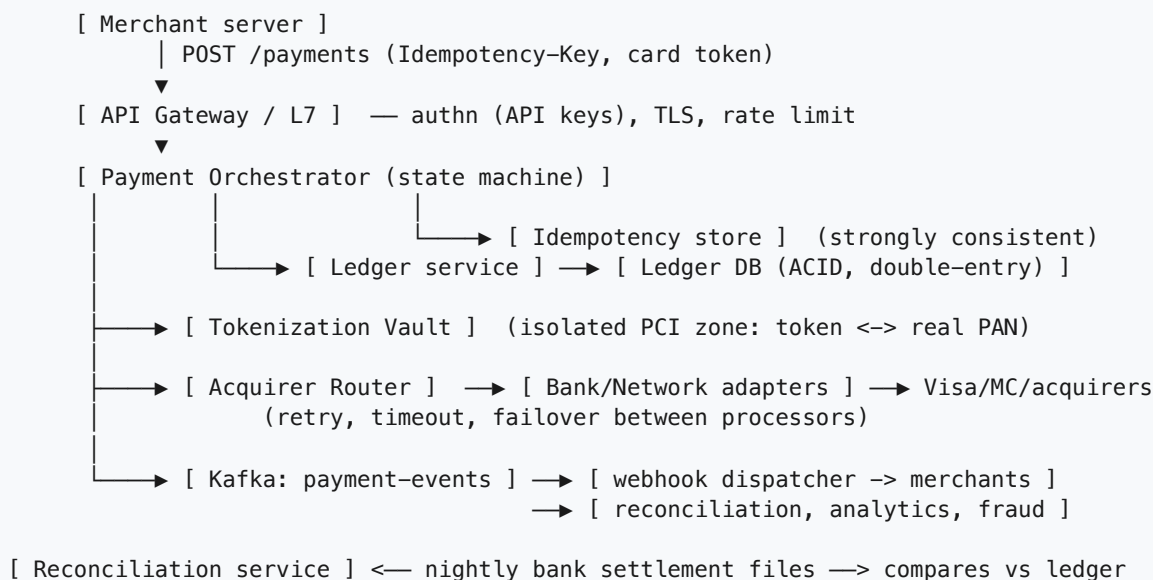
entry_id	account	debit	credit	txn_id	ts
e1	customer_receivable	100		t123	...
e2	merchant_payable		100	t123	...
		^^^ sum(debits) == sum(credits) ALWAYS			

Balances are DERIVED by summing entries, never mutated in place.

This makes every cent traceable and lets reconciliation prove the books balance.

Why double-entry: it's self-checking (debits must equal credits), immutable (append-only = perfect audit trail), and it's the centuries-old accounting model banks/auditors expect. A mutable "balance" column is unauditable and race-prone.

5. Architecture



- The **orchestrator** owns the state machine and calls the bank synchronously (within the buyer's patience window), writing ledger entries and idempotency records transactionally.
- The **vault** is a separate, hardened, PCI-scoped service; the rest of the platform handles only tokens, shrinking PCI audit scope dramatically.
- **Kafka** carries events out to webhooks, reconciliation, fraud, and analytics — none of those belong on the synchronous charge path.

6. Which datastore — and why (the detailed part)

Interviewer: What databases, and why? This is money — justify carefully.

Candidate: The core is unambiguously **a relational, ACID database** — this is the textbook case for it. I do not reach for eventual consistency anywhere near the ledger.

Concern	Choice	Why this, and why not the alternative
Ledger + transactions (source of truth)	Relational ACID DB (PostgreSQL / Amazon Aurora / CockroachDB)	Money demands ACID transactions, strong consistency, and constraints (balanced entries, unique idempotency keys). Double-entry + state transitions must commit atomically. <i>Not a NoSQL/eventually-consistent store</i> — a lost or reordered write here means real money lost or double-moved. CockroachDB/Spanner if we need horizontal scale with strong consistency across regions.
Idempotency store	Same ACID DB, unique constraint on (merchant, key) (or Redis with strong persistence as a fast front check)	Needs a hard uniqueness guarantee to block double-charge races. <i>Not a best-effort cache</i> — a missed dedup = double charge.
Card data (PAN)	Dedicated tokenization vault (isolated, encrypted, PCI-DSS zone; e.g. HSM-backed)	Keeps raw card numbers in ONE hardened place; everything else uses tokens → smaller PCI scope, smaller breach blast radius. <i>Not storing PANs in the app DB</i> — that puts the entire platform in PCI scope and is a breach nightmare.
Event backbone	Kafka	Durable, replayable stream for webhooks, reconciliation, fraud, analytics; decouples slow downstreams from the charge path. Replay is invaluable for re-deriving state. <i>Not synchronous calls</i> — a slow webhook must never delay a charge.
Bank/acquirer integration	Adapter services + a durable outbox	Each network/bank has its own protocol; adapters normalize them; an outbox pattern ensures "we recorded intent to call the bank" survives crashes so we can resolve unknown outcomes.
Analytics / reporting	Warehouse (OLAP), fed from Kafka	Keep heavy reporting off the transactional ledger DB.

Say-it-out-loud justification: "The ledger is a strongly-consistent ACID relational database because money requires atomic, balanced, auditable writes and hard uniqueness for idempotency — this is the one system where I refuse eventual consistency. Card data lives in an isolated tokenization vault to shrink PCI scope; Kafka carries everything non-critical (webhooks, reconciliation, fraud) off the synchronous path; and reconciliation against bank settlement files is the safety net that proves the books balance. For global scale with strong consistency I'd use Spanner/CockroachDB rather than sacrifice correctness for a NoSQL store."

7. The hardest problem — unknown outcomes from the bank

Interviewer: You call the acquiring bank to authorize. The call **times out**. You don't know if the charge happened. Now what?

Candidate: This is *the* payment problem, and it's why idempotency + reconciliation exist.

1. **Persist intent before calling.** Using an **outbox / write-ahead** approach, we record `AUTH_PENDING` with our idempotency key **before** the bank call. So even if we crash mid-call, we know a call was in flight.
2. **Send our idempotency key to the bank too.** Card networks support idempotency/retrieval references — so *retrying* the auth with the same key is safe: the bank returns the original result rather than charging again.
3. **On timeout, don't guess — reconcile.** We either (a) **query the bank** for that key's status ("did this go through?"), or (b) leave it `PENDING` and let the **reconciliation job** resolve it against the bank's settlement file. We **never** optimistically mark it captured or silently retry without the key.
4. **Deterministic resolution:** every transaction ends in a known terminal state, resolved by either an explicit status query or the daily settlement reconciliation. No transaction is ever "lost."

```
record AUTH_PENDING (durable) → call bank (idempotency key) —timeout→ ???  
├ status query to bank with the same key → AUTHORIZED / DECLINED  
└ else remains PENDING → nightly reconciliation vs settlement  
                        → forces every txn to a terminal state
```

8. Reconciliation — the financial safety net

Banks send **settlement files** (daily/periodic). A reconciliation service compares **our ledger** against **the bank's record**, transaction by transaction: - **Match:** confirm settled. - **We have it, bank doesn't:** unresolved auth — investigate/expire. - **Bank has it, we don't:** a charge we lost track of — repair the ledger, alert. - **Amount mismatch:** flag for manual review.

This is how financial systems guarantee correctness *despite* an unreliable external world — you don't trust any single online call; you **reconcile against the authoritative settlement**.

9. Consistency, multi-region, failure modes

- **Consistency: strong** on the ledger and idempotency store, always. Everything else (webhooks, analytics, fraud scoring) is eventual. This is the deliberate inversion of most systems in this playbook — here correctness dominates.
- **Multi-region:** for global scale with strong consistency, use a **globally-consistent DB** (Spanner/CockroachDB) or region-pinned ledgers (a payment is owned by one region's ledger, data-residency-aware for regulations like PSD2/GDPR). Never let two regions independently decide the same payment's outcome.

- **Availability stance:** if the ledger DB is unreachable, **fail closed** (reject the payment) rather than risk an unrecorded charge. A declined checkout is recoverable; a double-charge or lost charge is not.

Failure	What happens	Mitigation
Bank call times out	Unknown outcome	Idempotency key + status query + reconciliation → deterministic terminal state
Orchestrator crashes mid-txn	Partial state	Outbox + durable state machine → resume/resolve on restart; idempotency prevents re-charge
Ledger DB down	Can't record money	Fail closed (reject new payments); never charge without a durable record
One acquirer down	Some routes fail	Acquirer router fails over to an alternate processor
Webhook endpoint down	Merchant not notified	Kafka retains events; retry with backoff; merchant can poll status API
Duplicate settlement/refund	Double refund risk	Idempotency + ledger uniqueness + reconciliation catch it

10. Follow-up curveballs

Interviewer: Auth and capture are often separate (hold now, charge on ship). How model that?

Candidate: Two transitions: `authorize` places a **hold** (ledger entries reserving funds, state `AUTHORIZED`), and `capture` later moves the held funds (state `CAPTURED`). Auths **expire** if not captured (banks release the hold after N days) — a sweeper transitions expired auths to `VOIDED` and reverses the reservation entries. Partial captures (ship part of an order) capture less than the authorized amount, leaving the remainder to void.

Interviewer: How do you prevent fraud without slowing the charge?

Candidate: Fraud scoring runs **async off the Kafka event** for post-hoc analysis, but a **fast synchronous risk check** (velocity, blocklists, ML score from a low-latency service) can gate high-risk transactions before the bank call. Keep the synchronous check tight (tens of ms) and push heavy analysis async; block/flag/review based on score.

Interviewer: A refund needs to happen but the original settlement hasn't cleared yet.

Candidate: Refunds are their own state/entries against the original transaction. If the capture hasn't settled, a **void** (cancel the auth) is cheaper than a refund; if it has settled, issue a **refund** (new money movement back to the customer) with its own idempotency key and ledger entries. Reconciliation tracks both against settlement.

11. Deep-dive: "Why not just keep a balance column and update it?"

Interviewer: You insisted on an append-only double-entry ledger. Why not the obvious thing — a balance column you `UPDATE balance = balance + amount` per transaction? It's simpler.

Candidate: Because a mutable balance is unauditable, race-prone, and destroys history — three things you cannot give up when the data is money.

Mutable balance: <code>UPDATE accounts SET balance = balance + 100 WHERE id=X</code> -> the old value is GONE -> "why is the balance 4,217?" is unanswerable -> concurrent updates race on the row	Append-only ledger: <code>INSERT two balanced rows per movement (debit somewhere == credit elsewhere)</code> -> nothing is ever changed or deleted -> <code>balance = SUM(entries)</code> ; every cent traces to a specific event -> appends don't fight over one value
--	---

Concern	Mutable <code>balance</code> column	Append-only double-entry ledger
Auditability	History overwritten; can't prove how you got the number	Every movement is an immutable row; full trace
Concurrency	Two updates race on the same row (lost update / hot-row lock)	Independent appends; no contention on one value
Correctness check	None — a bug silently corrupts the balance	Debits must equal credits — self-checking invariant
Reversal / dispute	Overwrite again, losing the original	Post a compensating entry; original stays for audit
Regulatory	Fails audit — money systems must be traceable	This is the model banks/auditors expect

Why not trust a NoSQL/eventually-consistent store for this? The ledger needs ACID transactions (both legs of an entry commit atomically) and hard uniqueness (idempotency keys). An eventually-consistent write here means real money lost or double-moved — so the ledger is the one place I refuse eventual consistency, and use Postgres/Spanner/CockroachDB.

Rule of thumb: whenever the row is money, append immutable facts and derive the total — never overwrite the total in place.

12. Deep-dive: the dual-write problem — ledger commit vs. Kafka publish

Interviewer: You commit the payment to the ledger DB, then publish an event to Kafka for webhooks and reconciliation. What if the process crashes *between* those two writes? Walk me through it.

Candidate: This is the **dual-write problem**, and it's the sharpest correctness edge in the whole design. Two separate systems (the DB and Kafka) can't be updated in one atomic transaction, so a crash in the gap leaves them inconsistent.

```
t0 BEGIN; write CAPTURED + ledger entries; COMMIT;  <- money is now real in the DB
t1 <process crashes here>                           <- the gap
t2 (would have) published "payment.captured" to Kafka <- NEVER happened
```

Result: DB says captured, but no event → webhook never fires, reconciliation never sees it, downstream (fulfillment, emails) never runs.

The INVERSE (publish first, then DB write fails) is worse: an event says "captured" for money that was never actually recorded.

Ranked fixes:

1. **Transactional outbox (recommended).** In the *same* DB transaction that writes the ledger, insert a row into an `outbox` table. A separate relay process reads new outbox rows and publishes them to Kafka, marking them sent. Because the ledger write and the outbox write commit **atomically**, an event exists for every committed payment — the relay just might publish it slightly later (or twice, so consumers dedup). This turns two systems into one atomic write plus an at-least-once relay.

```
BEGIN; INSERT ledger entries ...; INSERT INTO outbox(event) VALUES ('payment.captured',
...); COMMIT; <- both or neither | [ relay ] polls outbox / reads the DB change-log →
Kafka → webhooks, recon └ marks the outbox row 'published' (retries safely; consumers
dedup by event id)
```

1. **Change Data Capture (CDC).** Instead of a relay polling the outbox, tail the DB's replication log (Debezium-style) and publish committed changes to Kafka. Same guarantee — the event derives from the *committed* DB state — with less custom code.
2. **Idempotent, deduping consumers.** Whichever of the above you pick, delivery is at-least-once, so every consumer dedups on event id. (True exactly-once across a DB and a broker is impossible; at-least-once + idempotency is the real-world answer.)
3. **Reconciliation as the backstop.** Even if events are lost entirely, the daily settlement reconciliation compares the ledger against the bank's record and repairs any gap — the ultimate safety net that doesn't depend on the event pipeline at all.

What to say out loud: "Committing to the DB and publishing to Kafka is a dual-write — a crash in the gap desyncs them. I'd use the transactional outbox: write the event to an outbox table in the same transaction as the ledger, so they commit atomically, then relay it to Kafka at-least-once with deduping consumers. CDC is the lower-code variant. And reconciliation against the bank settlement is the backstop that catches anything the pipeline drops — because for money, I always want a second, independent source of truth."

Summary — the mental model

A payment gateway is "move money correctly to the cent against a slow, unreliable external world, and be able to prove it." The pillars: **idempotency keys** on every mutation to kill double-charges; an

append-only double-entry ledger in a strongly-consistent ACID database as the auditable source of truth; a **durable state machine + outbox** so no crash loses a transaction; a **tokenization vault** to contain PCI scope; **Kafka** to push webhooks/fraud/ analytics off the critical path; and **reconciliation against bank settlement files** as the ultimate safety net that resolves every unknown outcome. Unlike every other system here, this one **chooses consistency over availability** — a rejected payment is recoverable; a double-charge or lost charge is not.